

physics-AI For CFD Simulations: SciML

October 6, 2026

Outline

- 1 Motivation
- 2 Physics-informed learning
- 3 Case study: crystal growth
- 4 Agentic CFD with AutoFOAM
- 5 Integration and outlook
- 6 References

cryo_1.jpg

Agenda

01 Motivation

Why CFD is expensive and where AI can help.

02 Physics-informed surrogates

PINNs $\rightarrow$ neural operators $\rightarrow$ geometry-aware models.

03 Evidence

Crystal-growth benchmarks: what improves accuracy, and what does not.

04 Agentic CFD

AutoFOAM: automate case set-up, execution and self-correction.

Integration, limitations and validation roadmap follow.

Glossary: terms used in this talk

ال ل ال ل

CFD Computational fluid dynamics: numerically solving fluid-flow equations.

PDE Partial differential equation, e.g. Navier–Stokes or heat equation.

PINN Neural network trained so its output satisfies the PDE. Residual How far a candidate solution is from satisfying the PDE at a point.

BC / IC Boundary / initial conditions (values imposed at edges / at $t = 0$).

Autodiff Automatic differentiation: exact derivatives of the network output.

Operator Map from a function (input field) to a function (solution). NRMSE RMSE divided by the standard deviation of the target field.

RANS Reynolds-averaged Navier–Stokes (time-averaged turbulence model).

OOD Out-of-distribution: test cases never seen in training.

RAG Retrieval-augmented generation (look up verified examples). SFT / DPO

Supervised fine-tuning / Direct Preference Optimisation.

CFD is accurate, but expensive

Why accelerate?

- **High cost per simulation:** high-fidelity, multi-physics CFD needs fine meshes and long runs.
- **The real cost is the sweep, not one run:** design studies vary pull rate, rotation rates, thermal boundary conditions, geometry... each point is a new simulation.
- **Multi-physics coupling** (flow + heat + particles) adds model complexity.
- **Set-up expertise:** a valid case needs deep solver knowledge.

cz-process-schematic
3.8cm Silicon
crystal-growth (CZ) process
schematic: crucible, melt, crystal,
rotation, heater

Goal: keep the governing physics, cut the cost, and make set-up accessible.

Application focus: two demanding CFD settings

Silicon crystal growth (Czochralski, CZ)

- A seed crystal is pulled from molten silicon in a rotating crucible.
- Couples heat transfer, melt convection, pressure variation and rotating boundaries.
- The resulting thermal and flow fields largely determine crystal quality.

cz-furnace-fields2.5cmCZ melt flow
/ temperature field

Cryogenic / hydrogen systems

- Design-oriented CFD for complex flow and thermal conditions.
- Faster surrogates support iterative design.
- Relevant to the wider space and energy use cases.

cryogenic-system2.5cmCryogenic
tank / hydrogen system CFD

What the CFD has to solve (CZ melt)

Governing physics in the reference model

- Incompressible Navier–Stokes: momentum + continuity.
- Energy equation for temperature T .
- **Boussinesq buoyancy**: density varies with T only in the buoyancy term, which drives natural convection.
- **Rotation**: crystal and crucible walls impose azimuthal velocity $u_\theta = \omega r$.

Unknowns per point (r, z)

u_r , u_z , u_θ (velocity), p (pressure), T (temperature): five coupled fields.

cz-domain-bcs4.9cm2D
axisymmetric melt domain
with boundary conditions:
rotating walls, slip free
surface, hot wall, crucible
bottom

The technical story

[node distance=0.5cm, box/.style=rounded corners,draw=teal,fill=mint,minimum width=2.9cm,minimum height=1.05cm,align=center,font=, arr/.style=-Latex[length=2mm],very thick,draw=orange] [box] (cfd) High-fidelity CFD; [box,right=of cfd] (pinn) Physics-informed neural networks; [box,right=of pinn] (op) Neural operators, geometry awareness; [box,right=of op,fill=lightorange,draw=orange] (ag) Agentic CFD automation; [arr] (cfd)–(pinn); [arr] (pinn)–(op); [arr] (op)–(ag); [below=0.4cm of cfd,align=center,text width=3.0cm,font=]Accurate but slow and expertise-heavy; [below=0.4cm of pinn,align=center,text width=3.0cm,font=]Encode governing equations and conditions in the loss; [below=0.4cm of op,align=center,text width=3.0cm,font=]Generalise across parameters, fields and geometries; [below=0.4cm of ag,align=center,text width=3.0cm,font=]LLM agent turns natural language into runnable OpenFOAM cases;

Research question: how can physics-aware surrogates and autonomous case generation together make CFD faster and more accessible, without

Physics-Informed Neural Networks (PINNs): the idea

Core idea

A neural network $\hat{u}_\theta$ represents the solution. Training penalises both **data mismatch** and **physics violations**.

- 1 Sample points in the domain and on its boundaries.
- 2 Use automatic differentiation to evaluate PDE residuals.
- 3 Optimise one objective combining PDE, BC and IC terms.

Why it is attractive

Physics enters training directly, reducing reliance on labelled solution data.

pinncpp3.6cmPINN concept:
coordinates $\rightarrow$ network $\rightarrow$ field;
autodiff gives residuals; loss =
PDE + BC + IC

Raissi, Perdikaris & Karniadakis

PINN methodology: physics becomes part of the objective

$$L_{\text{total}} = L_{\text{PDE}} + L_{\text{BC}} + L_{\text{IC}} + \lambda L_{\text{SAL}}$$

pinm-loss-diagram4.0cmPINN
loss composition / training
workflow

- L_{PDE} : governing-equation residual.
- $L_{\text{BC}}, L_{\text{IC}}$: boundary and initial-condition mismatch.
- λ : relative weighting of an additional term.

The central issue

The **weights matter**. A smaller weighted loss does not necessarily mean a more accurate solution.

Raissi et al. (2019); Wang, Teng & Perdikaris (2021)

Why PINN training is not automatically stable

Why optimisation can fail

- PDE and constraint terms can have very different gradient scales.
- Stiff or competing terms make the loss landscape difficult to optimise.
- A single aggregate loss can hide a badly satisfied constraint.

Implication for this study

Compare methods using:

- 1 each loss component;
- 2 an external solution error when a reference exists;
- 3 physical validation where possible.

pinn-training-
stability2.4cmlllustration of
competing loss terms / gradient
imbalance

Wang, Teng & Perdikaris (2021); Krishnapriyan et al. (2021)

From PINNs to neural operators

From one solution to a solution family

A PINN typically solves one PDE instance. A **neural operator** learns a map

inputs: coefficients / geometry / conditions $\longrightarrow$ solution field.

pino-architecture5.0cmPINO /
physics-informed operator: input
function $\rightarrow$ operator network $\rightarrow$
solution function, with physics loss

- One model can serve many parameter settings.
- Physics-informed operators add PDE constraints during training.
- This is better aligned with repeated-query CFD workflows.

Kovachki et al. (2023); PI-GANO, Zhong & Meidani (2024)

Geometry-aware operators: the next generalisation step

Why geometry matters

Real CFD design spaces change both **parameters** and **shape**.

Geometry-aware operators encode the domain rather than assuming one fixed grid.

- **PI-GANO**: physics-informed + geometry-aware operator learning.
- **GAOT / MAGNO**: attention / graph-based operator learning on irregular geometry.

CFD implication

The target is not one surrogate for one case, but a reusable solution map across operating conditions and geometries.

geometry-aware-
operators4.2cmPI-GANO / GAOT
architecture overview

Comparing the classical approaches

||||

Approach	What it learns	Main advantage	Main caveat
PINN	One PDE solution	Physics constraints	Training stability / scaling
PINO	Solution operator	Reusable across parameters	Needs representative coverage
PI-GANO	Geometry-aware operator	Parameter + geometry generalisation	Geometry encoding / validation
GAOT	Graph / attention operator	Irregular-domain modelling	Cost and validation

approach-comparison0.71.9cmOptional: comparison figure / benchmark chart

Kovachki et al. (2023); Zhong & Meidani (2024); GAOT/MAGNO (2025)

Case study overview: two experiments, two questions

A. Adaptive PINN training

Question: when loss weights adapt during training, is a lower PDE loss really a better solution?

Problems: (1) 1D heat equation with an analytical answer; (2) simplified Czochralski thermal-fluid PINN.

Compared: Baseline PINN, GNPINN, AgenticPINN.

B. CFD surrogates (Gaussian process)

Question: how well does a surrogate predict a *whole unseen operating condition*, field by field?

Data: ANSYS Fluent reference CFD, three one-parameter sweeps.

Metric: NRMSE per output field.

Shared message

Aggregate numbers hide where a model fails. Report component-level losses and field-wise errors.

Work with N. Chapagain, R. Raj, B. Kurisinkal Shine (NeurIPS 2026 Sim2Science workshop).

Case study: CFD reference data for Czochralski growth

Reference CFD

- ANSYS Fluent 2026 R1; **steady, laminar, 2D axisymmetric** melt.
- Incompressible Navier–Stokes + energy, Boussinesq buoyancy, azimuthal velocity from rotation.
- Rotating walls: $u_\theta = \omega r$; free surface non-penetrating and slip; prescribed thermal conditions at free surface and crucible bottom.
- Mesh-independence study $\Rightarrow$ common grid of 8181 nodes.
- Rotational Reynolds numbers are large, so these are *controlled reference cases*, not fully resolved industrial melt flow.

cz-cfd-reference4.6cmReference
CFD fields (temperature,
velocity components) in (r, z)

Three parameter sweeps and the case-wise holdout

One-parameter sweeps

Parameter	Range	Held-out
T_{hot} (hot wall)	1730–1785 K	1780 K
ω_c (crucible)	–10 to –1 rpm	–4 rpm
ω_s (crystal)	4–20 rpm	7 rpm

Surrogate learns

$$(r, z, s) \mapsto (u_r, u_z, u_\theta, p, T)$$

s is the active process parameter of the sweep.

Why a case-wise holdout?

One *entire operating condition* is withheld for testing. This measures generalisation to an unseen process setting, not interpolation between randomly split spatial points of cases already seen.

holdout-schematic2.6cmSweep
values on a line; one value held
out for test

Benchmark 1: the 1D heat equation (analytical reference)

$$\frac{\partial T}{\partial t} = \alpha \frac{\partial^2 T}{\partial x^2}, \quad T(0, t) = T(1, t) = 0, \quad T(x, 0) = \sin(\pi x)$$

heat-equation-
exact4.8cmAnalytical
heat-equation solution $T(x, t)$
over $[0, 1]^2$

$$T_{\text{exact}}(x, t) = \sin(\pi x)e^{-\alpha\pi^2 t}$$

- $\alpha = 1$ in the benchmark.
- Exact solution gives a direct **field-accuracy** metric.
- This is the control experiment for interpreting PINN losses.

Important

The crystal-growth case has no analytical reference; its losses are constraint diagnostics, not proof of accuracy.

Adaptive PINN training: what do we compare?

Matched comparison

- **Baseline PINN**: fixed unit weights.
- **GNPINN**: normalises PDE and BC/IC gradients.
- **AgenticPINN**: rule-based controller adapts the weights.

training-strategies-
overview4.0cmThree trainers:
fixed weights / normalised
gradients / adaptive
controller

$$\mathcal{L} = \lambda_{\text{PDE}} \mathcal{L}_{\text{PDE}} + \lambda_{\text{BC/IC}} \mathcal{L}_{\text{BC/IC}}$$

Fair comparison

Same architecture, optimiser, sampling budget, seed and learning-rate schedule within each experiment.

The three strategies in detail

Baseline PINN

Fixed unit weights; minimise the combined PDE + BC/IC loss.

GNPINN

Compute PDE and BC/IC gradients separately, normalise them, then combine.

AgenticPINN

Monitor gradient norms and update the two loss weights with bounded, smoothed rules.

What changes?

The network and physics stay fixed; only the **training objective / gradient balancing** changes.

AgenticPINN: the controller rule

With smoothed gradient norms $\bar{g}_{\text{PDE}}$ and $\bar{g}_{\text{BC/IC}}$:

$$a = \text{clip}\left(\sqrt{\frac{\bar{g}_{\text{PDE}} + 10^{-12}}{\bar{g}_{\text{BC/IC}} + 10^{-12}}}, 0.8, 1.25\right)$$

$$\lambda_{\text{PDE}} \leftarrow \text{clip}(\lambda_{\text{PDE}}/a), \quad \lambda_{\text{BC/IC}} \leftarrow \text{clip}(\lambda_{\text{BC/IC}}a)$$

- Weight changes are bounded; weights remain in $[10^{-2}, 10^2]$.
- A learning-rate reduction is triggered when progress stalls.

Intuition

Balance the **gradient influence** of PDE and constraints.

controller-flow2.6cmController
loop: measure gradient norms
→ update weights → train

Why the weighted total is not a fair score

Three different questions

- 1 Did the optimiser reduce the chosen objective?
- 2 Are all physical constraints satisfied?
- 3 Is the predicted field accurate?

$$\mathcal{L} = \sum_k \lambda_k \mathcal{L}_k$$

If λ_k changes between methods, **weighted totals are not directly comparable**.

Observed example

AgenticPINN ends with $\lambda_{\text{PDE}} = 10^2$ and $\lambda_{\text{BC/IC}} = 10^{-2}$.

$$\mathcal{L}_{\text{total}} \approx 2.08 \times 10^{-3}$$

yet the heat-equation relative L_2 error is **1.368**.

Result: low PDE residual $\neq$ accurate solution

|| $\hat{X}$ $\hat{X}$ $\hat{X}$

Problem	Trainer	PDE loss	BC/IC loss	Rel. L_2 error
---------	---------	----------	------------	------------------

Heat	Baseline	3.96×10^{-3}	1.15×10^{-3}	8.43×10^{-2}
------	----------	-----------------------	-----------------------	-----------------------

Heat	GNPINN	9.74×10^{-3}	9.05×10^{-4}	5.41×10^{-2}
------	--------	-----------------------	-----------------------	-----------------------

Heat	AgenticPINN	8.22×10^{-7}	2.00×10^{-1}	1.368
------	-------------	-----------------------	-----------------------	-------

Crystal	Baseline	9.26×10^{-6}	1.63×10^{-5}	–
---------	----------	-----------------------	-----------------------	---

Crystal	GNPINN	1.61×10^{-5}	1.01×10^{-4}	–
---------	--------	-----------------------	-----------------------	---

Crystal	AgenticPINN	1.14×10^{-6}	1.29×10^{-2}	–
---------	-------------	-----------------------	-----------------------	---

Main result

The smallest PDE residual is **not** the best field solution.

Report instead

PDE loss + BC/IC loss + external error
whenever a reference exists.

Heat equation: reading the numbers

AgenticPINN

- PDE residual $\downarrow \approx 4.8 \times 10^3$ vs baseline.
- BC/IC loss $\uparrow \approx 173\times$.
- Relative L_2 error = **1.368**.

Best field accuracy

GNPINN achieves the lowest relative L_2 error: 5.41×10^{-2} .

heat-mlp-method-comparison4.0cmAnalytical vs
Baseline / GNPINN /
AgenticPINN temperature fields,
common axes and colour scale Rel.
 L_2 : baseline 8.43×10^{-2} ; GNPINN
 5.41×10^{-2} ; AgenticPINN 1.368.

Simplified crystal-growth PINN: same trade-off

Observed trade-off

AgenticPINN reduces the PDE residual by $\approx 8.1\times$, but BC/IC loss rises by $\approx 792\times$.

Interpretation

No analytical solution is available, so these numbers show **constraint satisfaction**, not verified field accuracy.

crystal-mlp-fields4.0cmVelocity,
pressure, temperature at $t = 0.20$
for Baseline / GNPINN /
AgenticPINN Qualitative
comparison only; not validated
against industrial CFD.

Why it happens: controller dynamics

Observed mechanism

In both experiments the controller drives

$$\lambda_{\text{PDE}} \rightarrow 10^2, \quad \lambda_{\text{BC/IC}} \rightarrow 10^{-2}.$$

The resulting weight ratio is 10^4 .

controller-dynamics4.8cmAdaptive
loss-weight dynamics: λ_{PDE} and
 $\lambda_{\text{BC/IC}}$ vs epoch (crystal growth
left, heat equation right)

Lesson

Adaptive weighting can improve one objective while weakening another.

Control must be paired with external validation.

CFD surrogate: Gaussian-process set-up

Model

- Multi-output GPR for $(r, z, s) \mapsto (u_r, u_z, u_\theta, p, T)$, inputs and targets scaled.
- Kernel: constant $\times$ anisotropic RBF (one length scale per input) + white noise (10^{-4}); extra diagonal term 10^{-6} .
- Kernel hyperparameters are fixed, not optimised during fitting.
- Returns predictive mean and standard deviation, mapped back to physical units.

Metric

$$\text{NRMSE} = \frac{\sqrt{\frac{1}{N} \sum_i (\hat{y}_i - y_i)^2}}{\sigma_y}$$

σ_y : standard deviation of that field in the holdout case. Normalisation lets us compare fields with different units and scales.

Uncertainty diagnostic

Pearson correlation between $|\hat{y} - y|$ and predictive std; an error-awareness check, not a formal calibration.

Result: CFD surrogate accuracy depends on field and sweep

Case-study result; see the references slide for supporting literature.

GP surrogates, held-out operating condition (NRMSE)

Temperature field, by sweep	NRMSE
Hot-wall temperature	5.61%
Crucible rotation	6.34%
Crystal rotation	9.58%
Crystal-rotation sweep, by field	NRMSE
Swirl velocity u_θ	2.93%
Radial velocity u_r	22.95%
Axial velocity u_z	41.46%

surrogate-nrmse-
overview3.0cmField-wise NRMSE
for the three holdouts

Message

Temperature is predicted well; meridional velocities (u_r, u_z) under crystal rotation are not, while swirl u_θ is predicted well (2.93%). Aggregate scores hide this.

Case study: what we learn

1. Objective $\neq$ accuracy

Adaptively weighted objectives are optimisation quantities, not common cross-method accuracy metrics.

2. Report components

Unweighted PDE, BC and IC losses, plus external solution error where available.

3. Report per field

Surrogate error varies strongly by field and by sweep; pooled scores conceal it.

Scope of the claims

Matched comparisons use **one initialisation and two physics settings**; the conclusions should not be read as universal across adaptive-weighting methods or PDEs. CFD cases are steady, laminar and axisymmetric. Future work: more PDEs, random seeds, adaptive strategies and higher-fidelity crystal-growth models.

The OpenFOAM expertise barrier

OpenFOAM workflow context; related agentic-CFD work listed on the references slide.

Why CFD is hard to access

- OpenFOAM is powerful and free, with unmatched flexibility.
- Each case needs a dozen or more interlinked **C++ dictionaries**, a mesh and the right solver.
- Syntax is **strict and deeply interconnected**: one inconsistent entry stops the run.
- Going from physics idea to working simulation is slow, error-prone and needs scarce expertise.

Where generic LLMs fall short

- Invent non-existent OpenFOAM keywords.

The vision

“Simulate a turbulent flow over a NACA0012 airfoil at $Re = 10^6$ ”



Full, convergent OpenFOAM case

generated autonomously

openfoam-case-
structure1.7cmTypical
OpenFOAM case tree: 0/,
constant/, system/ dictionaries

Related work: FoamGPT,
OpenFOAMCPT, Foam Agent

AutoFOAM at a glance

What it is

A **self-improving AI agent** (fine-tuned 14B model) that turns a plain-English request into a meshed, configured and *running* OpenFOAM v2412 simulation.

Scope

- **7 solver families, 13 mesh templates** (gmsh).
- No manual CAD or case editing.
- Output is a standard OpenFOAM case you can inspect and edit.

Example request

"NACA0012 airfoil, AoA 5°, $Re = 10^6$, chord = 1 m"

Practicalities

- Runs **on-premise**: your data stays with you.
- Self-hosted: one GPU (≥ 24 GB) + OpenFOAM v2412.
- Python API, interactive REPL and batch runs.

AutoFOAM: the eight-stage pipeline

```
[ font=, stage/.style=rounded corners,draw=teal,fill=mint,minimum
width=2.7cm,minimum height=0.9cm,align=center,inner sep=2pt, llm/.style=rounded
corners,draw=orange,fill=lightorange,minimum width=2.7cm,minimum
height=0.9cm,align=center,inner sep=2pt,
arr/.style=-Latex[length=2mm],thick,draw=midgray,
retry/.style=-Latex[length=2mm],thick,dashed,draw=alertred ] [llm] (n1) at (0,0) 1.
Prompt; [llm] (n2) at (3.4,0) 2. Parameters; [stage] (n3) at (6.8,0) 3. Solver; [stage]
(n4) at (10.2,0) 4. Mesh; [stage] (n5) at (10.2,-1.5) 5. Numerics; [stage] (n6) at
(6.8,-1.5) 6. Case; [stage] (n7) at (3.4,-1.5) 7. Run; [stage] (n8) at (0,-1.5) 8. Score;
[arr](n1)-(n2); [arr](n2)-(n3); [arr](n3)-(n4); [arr](n4)-(n5); [arr](n5)-(n6);
[arr](n6)-(n7); [arr](n7)-(n8); [retry](n8.west)-++(-0.6,0)--(n2.north);
[font=,text=alertred] at (3.5,-2.55) retry after failed / low-scoring runs;
```

LLM

Understands intent and extracts parameters.

Deterministic code

Selects solver, writes the case and executes OpenFOAM.

Feedback

Scores the run and uses failures to trigger correction.

Pipeline walk-through: stages 1–4 (understand and mesh)

ic il L

Stage Output

- 1 Prompt refine Explicit, normalised CFD request.
 - 2 Parameter extraction Schema-constrained geometry, Re , fluid and solver hints.
 - 3 Solver selection Deterministic route based on transient / compressible / thermal / multiphase flags.
 - 4 Mesh Parameterised gmsh template with consistent OpenFOAM boundary patches.
-

Design principle

Use the model for **language**; use deterministic rules for decisions that affect physics or execution.

Pipeline walk-through: stages 5–8 (configure, run, judge)

ic il L

#	Stage	Output
---	-------	--------

- | | | |
|---|------------------|--|
| 5 | Numerical policy | Flow-regime-consistent schemes and turbulence settings. |
| 6 | Case writing | Consistent OpenFOAM dictionaries and boundary conditions. |
| 7 | Run | Solver logs, residuals and execution status. |
| 8 | Score | Convergence, residuals, solver choice, conservation and BC checks. |
-

Retry

Low score, divergence or NaN → inspect the solver feedback, correct the case, and rerun.

Key design choices

Constrained parameter extraction

- **vLLM** for high-throughput inference.
- **xgrammar** JSON schema enforces the `CFDParams` structure, so output is always parseable.
- Output: geometry, Re , fluid, solver, turbulence model.
- Regex check of $Re = UL/\nu$ consistency between stated and derived values.

Deterministic solver routing

- Decouples solver choice from the LLM.
- Physics flags: transient, compressible, thermal, multiphase.
- Routes to one of **7 approved solvers**.
- Avoids LLM bias, e.g. `simpleFoam` for transient vortex shedding.

Design principle

Use the LLM for language understanding; use rules and code wherever a wrong answer would crash a run or give a wrong physics set-up.

The reward: how a run is scored

Positive terms

Criterion	Weight
Convergence	0.40
Residual $< 10^{-4}$	0.20
Correct solver	0.10
Mass conservation	0.05
Valid boundary conditions	0.05

Penalties (up to -0.25)

Poor mesh quality, slow runtime, stagnation.

How the score is used

- $r < 0.7$: trigger a retry.
- $r \geq 0.5$: trajectory accepted into the foundational corpus.
- $r \geq 0.65$: live run joins the training pool.

Known bias

Reward favours convergence; a converged case can still be physically wrong (see limitations).

Training data: static corpus + dynamic capture

Foundational knowledge corpus

- **402 rows** of expert OpenFOAM trajectories.
- 252 unique prompts across all solver types and geometries.
- Accepted trajectories: score $r \geq 0.5$.

Dynamic trajectory capture

- Live production runs feed the dataset continuously.
- High-success runs ($r \geq 0.65$) join the training pool.
- Currently **210 de-duplicated topologies**.
- Failed runs logged by type for DPO mining.

Out-of-distribution benchmark: 110 prompts (isolated from training)

Solver family	Count	Share
simpleFoam (steady incompressible)	74	67%
rhoSimpleFoam (steady compressible)	9	8%
interFoam (multiphase VOF)	7	6%
Others (pimpleFoam, ...)	20	18%

The seven-layer self-evolution loop

[Small, illegible text line]

Guardrail

The loop only promotes a new model after held-out evaluation and regression checks.

The seven layers in plain language

jc j| L L

L	Name	What it does	Risk it addresses
---	------	--------------	-------------------

- | | | | |
|---|------------------|--|--------------------------|
| 1 | Self-correct | Fixes failed runs from solver logs and retries. | Wasted failed runs |
| 2 | Curate | Keeps only high-scoring ($r \geq 0.65$), de-duplicated cases. | Learning from bad data |
| 3 | Fine-tune + gate | One corrective SFT epoch, then test on held-out OOD prompts. | Shipping a worse model |
| 4 | DPO | Learns from fail→fix pairs: rejected failure vs chosen correction. | Repeating known mistakes |
| 5 | Anchor mix | 30% original corpus in every batch. | Catastrophic forgetting |
| 6 | Active learning | Focuses on the weakest solver family; generates new seeds. | Uneven solver coverage |
| 7 | Regression check | Diff against a pinned baseline before promotion. | Silent regressions |
-

Anti-collapse mechanisms

The self-distillation risk

Training on self-generated data can quickly degrade quality. AutoFOAM uses **three complementary defences** plus a preference-learning layer.

RAG-augmented retry

Chroma vector DB of verified OpenFOAM tutorial configs; relevant dictionary snippets are injected into the retry prompt, grounding the fix in known-good files.

Surgical dictionary patching

On localised FOAM FATAL errors only the offending dictionary is patched, skipping costly `gmsH` remeshing.

Prompt-diversity paraphrasing

Varies the phrasing of training prompts so the model does not overfit one pattern.

Layer 4: Direct Preference Optimisation (DPO)

Failure trajectory (*rejected*) $\longleftrightarrow$ Layer-1-corrected success (*chosen*)
DPO explicitly penalises the formulations that caused divergence.

Results: autonomous case generation

Coverage

- **7 solver families**, e.g. simpleFoam, pimpleFoam, rhoSimpleFoam, interFoam, icoFoam.
- **13 gmsh geometries**: cavity, step, NACA, cylinder, periodic hill, S-bend, Ahmed body, ...
- Input: one natural-language sentence.
- Output: correct polyMesh boundaries and a convergent solution.

OOD headline metrics (110 prompts)

Metric

FOAM FATAL-free	11
Execution success	
Solver routing accuracy	96.4% (10)
Mean reward r	
Geometry templates	
Solver families	

Misrouted: 4/110

Ambiguous low- Re cases only, e.g. transient cylinder at $Re = 250 \rightarrow$ icoFoam (physically justifiable).

Representative OOD prompts

lll c

Tag	Prompt	Predicted solver	Match
-----	--------	------------------	-------

cav_re150	Cavity flow $Re=150$ in water, 0.4 m square	simpleFoam	✓
-----------	---	------------	---

pipe_lam	Laminar pipe flow $Re=500$, diameter 0.02 m, water	simpleFoam	✓
----------	---	------------	---

naca_re1m	NACA0012 at $Re=10^6$, 5° angle of attack	simpleFoam	✓
-----------	---	------------	---

dam_tall	Tall dam break, 2 m wide, 5 m water column collapse	interFoam	✓
----------	---	-----------	---

cdnoz_ma04	Convergent-divergent nozzle, Mach 0.4 throat, air	rhoSimpleFoam	✓
------------	---	---------------	---

cyl_re250	Transient cylinder $Re=250$ in water, $D=0.1$ m	icoFoam	×
-----------	---	---------	---

Evaluation infrastructure

8-way sharded cluster of **NVIDIA H100** GPUs; 110 OOD prompts kept isolated from training data.

Autonomously generated meshes and flow solutions

mesh- airfoil1.9cmNACA mesh vel- airfoil1.9cmNACA $ U $ NACA 4-digit airfoil	mesh- cylinder1.9cmcylinder mesh vel- cylinder1.9cmcylinder $ U $ Cylinder in cross-flow	mesh-periodic- hill1.9cmperiodic hill mesh vel-periodic- hill1.9cmperiodic hill $ U $ Periodic hill	mesh- sbend1.9cmS- bend mesh vel-sbend1.9cmS- bend $ U $ S-bend duct
--	--	---	---

Top row: meshes. Bottom row: velocity magnitude. Each case came from a single natural-language prompt, with no manual intervention.

demo-thumbnail4.1cmVideo thumbnail /
screenshot of the AutoFOAM demo (click
opens YouTube)

What the demo shows

- Natural-language prompt input.
- Automated parameter extraction.
- gmsh mesh generation.
- OpenFOAM case file synthesis.
- Live solver run and residual convergence.
- Field contours and one-click ParaView.

Watch online

youtu.be/Kx17bfFlMSc

What you get, and how to work with us

What you get

- Hours of set-up cut to one prompt.
- Standard OpenFOAM cases you can inspect and edit.
- Python API, interactive REPL and batch runs.
- Self-hosted: one GPU (≥ 24 GB) + OpenFOAM v2412.

Extensible; work with us

The workflow can be extended to other CFD tools and to geometries of your choice.

Pilots, custom solver/geometry extensions and commercial licensing.

github.com/AGN000/AutoFOAM

How the pieces fit together (proposed workflow)

[Illegible text]

Generate

Agentic automation lowers the CFD set-up cost.

Accelerate

Verified CFD data supports reusable surrogates.

Validate

Independent physics checks prevent silent failure.

Limitations and validation gaps

AutoFOAM

- 13 parameterised geometries; no raw CAD pipeline.
- RANS-focused; broader physics remains to be tested.
- Reward favours execution / convergence and can miss physical error.

Physics-informed surrogates

- Small benchmark set and controlled initialisation.
- Crystal-growth case lacks a ground-truth field.
- Field difficulty is uneven: velocity can be much harder than temperature.

Priority

Add independent benchmark checks and field/component-level metrics before claiming general CFD accuracy.

Research and validation roadmap

Measure

Accuracy, conservation,
constraint satisfaction,
runtime.

Broaden

More geometries,
turbulent and
multi-physics
benchmarks.

Close the loop

Feed validator outcomes
back into agent training.

Key takeaways

- 1 CFD is the bottleneck: high-fidelity simulation is expensive and expertise-heavy.
- 2 Physics helps, but metrics matter: PINNs can reduce PDE residuals while producing worse fields.
- 3 Operators scale the idea: learn reusable maps across parameters and, increasingly, geometry.
- 4 Surrogate error is field-specific: temperature can be accurate while velocity remains difficult.
- 5 Agents remove set-up friction: AutoFOAM automates case generation, execution and correction.
- 6 Validation is the common layer: neither a low loss nor a converged CFD run is sufficient on its own.

Physics + geometry + agentic automation
→ accelerated CFD with explicit validation

Selected references

- M. Raissi, P. Perdikaris, G. E. Karniadakis (2019), *Physics-informed neural networks*, J. Comput. Phys. 378, 686–707. doi:10.1016/j.jcp.2018.10.045.
- S. Wang, Y. Teng, P. Perdikaris (2021), *Understanding and mitigating gradient flow pathologies in PINNs*, SIAM J. Sci. Comput. 43, A3055–A3081. doi:10.1137/20M1318043.
- A. Krishnapriyan et al. (2021), *Characterizing possible failure modes in PINNs*, NeurIPS 34.
- N. Kovachki et al. (2023), *Neural Operator: Learning Maps Between Function Spaces With Applications to PDEs*, JMLR 24.
- W. Zhong, H. Meidani (2024), *Physics-Informed Geometry-Aware Neural Operator*, arXiv:2408.01600.
- D. Kolberg, C. Anders, S. Reke (2023), *silicon VMCz crystal-growth PINNs*, doi:10.1080/00219592.2023.2236656.
- N. Chapagain et al. (2026), *Component-Level Evaluation of Adaptive PINN Training for CFD-Oriented Crystal Growth Simulation*, NeurIPS 2026 Sim2Science workshop.
- A. G. Neelan, A. Seshaditya, *AutoFOAM: The Self-Refining Autonomous OpenFOAM Agent*.

Project resources: AutoFOAM, FoamAgentCases, foam-cfd-unified-14b.

[remember picture,overlay] [navy] (current page.south west) rectangle (current page.north east); [teal] (current page.south east) rectangle ([xshift=-0.20]current page.north east); [orange] ([xshift=-0.20]current page.south east) rectangle ([xshift=-0.195]current page.north east);